FORM-TL-01
Notas de presentación: Vecindario

APUNTE PAZMIÑO WASHINGTON GERMAN
LOOR SUAREZ JORDAN JOSUE
MACIAS VASQUEZ CESAR ABRAHAN

Periodo académico 2026–2027

1. Contexto del escenario

Vecindario es una red social ficticia orientada a comunidades de barrio en ciudades intermedias de Ecuador.

La plataforma permite publicar avisos, seguir a vecinos y negocios locales, reaccionar, comentar y mantener conversaciones mediante mensajes directos.

2. Arquitectura propuesta

La arquitectura propuesta para Vecindario se organiza mediante servicios independientes que se encargan de las diferentes funciones de la red social. Esta separación permite distribuir el procesamiento de las publicaciones, la mensajería, las notificaciones y la moderación.

Los componentes principales son:

- **Cliente (CLI):** aplicación móvil o web desde la que los usuarios acceden a la plataforma, publican avisos y utilizan sus funcionalidades.
- **Servicio de publicaciones (PUB):** permite crear, editar y eliminar avisos, incluyendo publicaciones con imágenes.
- **Servicio de seguidores (SEG):** administra las relaciones entre usuarios, negocios locales y sus seguidores.
- **Servicio de muro (MUR):** organiza y distribuye las publicaciones que aparecen en el muro de cada usuario.
- **Servicio de mensajería directa (MSG):** administra las conversaciones entre usuarios y permite conocer su estado de conexión.
- **Servicio de moderación (MOD):** recibe las denuncias y gestiona el ocultamiento de publicaciones que incumplen las normas.
- **Servicio de notificaciones (NOT):** comunica a los usuarios los avisos y eventos relevantes de la plataforma.

Además, intervienen los siguientes actores externos:

- **Usuario:** utiliza la red social para publicar avisos, consultar el muro e intercambiar mensajes.
- **Negocio local:** publica información de sus productos, servicios y actividades para llegar a sus seguidores.
- **Moderador:** revisa las denuncias y participa en el proceso de moderación.
- **Servicio externo de almacenamiento de imágenes:** conserva las imágenes utilizadas en las publicaciones.

La arquitectura también contempla componentes de infraestructura para gestionar los eventos, conservar los mensajes, consultar la presencia de los usuarios y distribuir las conexiones de mensajería.

3. Mecanismos de comunicación

En Vecindario no todos los servicios necesitan comunicarse de la misma manera. Por ejemplo, enviar un mensaje directo requiere una respuesta rápida, mientras que distribuir una publicación entre miles de seguidores puede realizarse en segundo plano. Por esta razón, se han seleccionado distintos mecanismos de comunicación según las necesidades de cada operación.

3.1. Sockets TCP para mensajería directa

Para los mensajes directos se propone utilizar conexiones persistentes mediante WebSocket seguro (WSS), que funciona sobre TCP. Esto permite mantener una conexión abierta entre el cliente y el servicio de mensajería (MSG), evitando tener que iniciar una nueva conexión cada vez que se envía un mensaje [1].

De esta manera, los usuarios pueden intercambiar mensajes sin tener que esperar a que se realicen nuevas solicitudes constantemente. También permite actualizar el indicador de conexión para mostrar quién está en línea.

Para evitar que los mensajes aparezcan desordenados, MSG asignará un número de secuencia a cada mensaje dentro de su conversación. Además, los mensajes se guardarán antes de confirmar su recepción. Se utilizarán identificadores únicos para reconocer los mensajes y evitar que se procesen dos veces cuando sea necesario realizar un reintento.

Esta elección busca cumplir el requisito R2, que establece que los mensajes deben llegar en menos de 500 milisegundos cuando el destinatario está conectado y conservar el orden en que fueron enviados.

3.2. RPC mediante gRPC

Para la comunicación entre el servicio de muro (MUR) y el servicio de seguidores (SEG) se propone utilizar gRPC.

Por ejemplo, cuando una publicación debe aparecer en el muro de los seguidores, MUR necesita consultar a SEG para conocer qué usuarios siguen a la persona o negocio que

realizó la publicación. Para obtener esa información, MUR realiza una llamada RPC y espera la respuesta de SEG.

gRPC utiliza HTTP/2 para transportar los datos y Protocol Buffers para definir y serializar los mensajes que intercambian los servicios [2].

En este caso, la comunicación es síncrona porque MUR necesita recibir la respuesta de SEG antes de continuar con la operación que depende de esa información.

Se eligió gRPC porque permite establecer una comunicación directa y organizada entre los servicios, facilitando el intercambio de datos sin necesidad de utilizar mensajes de texto más extensos.

3.3. Mensajería asíncrona mediante Kafka

Para distribuir las publicaciones y comunicar eventos entre los servicios PUB, MUR y NOT se propone utilizar Apache Kafka.

La idea es que, cuando PUB registra una nueva publicación, envíe un evento a Kafka. A partir de ese momento, los servicios que necesitan procesar esa información pueden recibirla y trabajar por separado, sin que PUB tenga que esperar a que todos terminen.

Esto resulta útil porque una publicación puede llegar a miles de seguidores y no sería conveniente que el usuario tuviera que esperar durante todo ese proceso.

Kafka permite conservar los eventos para que puedan procesarse posteriormente si alguno de los servicios presentan problemas [3]. Cuando el servicio vuelve a funcionar, puede continuar procesando los eventos pendientes. También se deben utilizar mecanismos de idempotencia para evitar que una misma publicación se procese varias veces.

Para cumplir el requisito R3, se plantea dar prioridad a las alertas de seguridad, de modo que su distribución no tenga que esperar a que se terminen de procesar las publicaciones normales.

Esta solución también ayuda a responder al requisito R4, ya que permite distribuir las publicaciones de negocios con muchos seguidores de forma progresiva, sin concentrar todo el trabajo en un solo servicio.

3.4. Publicación y distribución al muro

Recorrido de un aviso. El usuario o el negocio local crea el aviso desde el cliente (CLI). La primera llamada (S1) envía a PUB el `id_usuario`, el texto, la categoría, la ubicación y el tipo de aviso (normal o alerta); PUB lo registra y responde con el `id_aviso`. Si el aviso lleva imagen, el cliente la sube directamente al servicio externo de almacenamiento (S2), que devuelve la URL o el identificador del archivo, y después registra esa referencia en PUB (S2b). Así el archivo no pasa por los servicios internos.

Cuando el aviso está completo, PUB publica el evento `AvisoPublicado` en Kafka (A1) y no espera nada más: el usuario ya recibió su confirmación. MUR consume el evento (A2) con un *planificador*, que pide a SEG los seguidores del autor por páginas mediante gRPC (S3) y publica un lote por página (`id_aviso`, número de página) en el tópico `muro.lotes`. Los *trabajadores* de MUR toman esos lotes en paralelo y agregan el aviso al muro de cada seguidor. En paralelo, NOT consume el evento `NuevaPublicación` o `AlertaSeguridad` (A3) y envía la notificación push a los seguidores (A5). Cuando un seguidor abre la aplicación, obtiene de MUR su muro ya armado (S10).

Mecanismos y justificación.

- **REST sobre HTTPS, síncrono (S1, S2, S2b).** Crear el aviso, subir la imagen y vincularla son acciones en las que el usuario necesita una respuesta inmediata (`id_aviso` o URL) para continuar. HTTPS funciona de forma nativa en Android y en el navegador y protege los datos en tránsito.
- **Kafka, asíncrono (A1, A2, A3).** La distribución al muro y las notificaciones pueden completarse después de responder al usuario. Kafka desacopla a PUB de MUR y de NOT: PUB publica el evento y continúa, y el evento queda guardado aunque un consumidor esté caído.
- **gRPC sobre HTTP/2, síncrono (S3).** MUR necesita cada página de seguidores para seguir distribuyendo. gRPC trabaja con mensajes binarios compactos y un contrato `.proto` entre servicios internos, por lo que tiene menos latencia que REST con JSON.

R3: tiempos distintos para avisos normales y alertas. Tomamos los tiempos de R3 como objetivos de diseño: hasta 5 s para un aviso normal, con consistencia eventual aceptable, y menos de 1 s para una alerta de seguridad. Para que una alerta no espere detrás de los avisos normales, PUB la publica en un tópico propio (`alertas.seguridad`), que leen grupos de consumidores dedicados en MUR y en NOT. Si se acumulan avisos normales en `avisos.publicados`, esa cola no retrasa a las alertas, porque tienen su propio tópico y su propia capacidad de procesamiento.

R4: un negocio con 30.000 seguidores. PUB no espera a que el aviso llegue a los 30.000 seguidores, porque la distribución es asíncrona (A1). Un evento de Kafka lo procesa un solo consumidor del grupo, así que no se reparte solo entre varios. Por eso el planificador de MUR lo divide: pide los seguidores a SEG por páginas (S3) y publica un lote por página en `muro.lotes`. Ese tópico tiene varias particiones, y los trabajadores de MUR, que forman un mismo grupo de consumidores, se reparten los lotes y los procesan en paralelo. Además, se limita cuántos lotes de un mismo autor se procesan a la vez, para que una publicación masiva no ocupe toda la capacidad y el servicio siga respondiendo al resto de usuarios.

F2: el servicio de muro se cae mientras un negocio publica. La distribución no se pierde, porque la unidad que se confirma y se reintenta es el lote. Cada trabajador confirma su lote en Kafka (*commit* del *offset*) solo después de escribir el aviso en todos los muros de ese lote, y el planificador confirma el `AvisoPublicado` solo cuando ya publicó todos los lotes. Si MUR se cae a mitad del trabajo, al recuperarse se vuelven a leer únicamente los lotes que no se confirmaron. Como un lote puede procesarse dos veces, la escritura es idempotente: agregar el mismo aviso al mismo muro no crea un duplicado, gracias a la clave única (`id_usuario`, `id_aviso`).

Socket TCP para la práctica PE-U1 (propuesta del equipo). WSS funciona sobre TCP, pero añade el protocolo WebSocket encima. Para dejar clara la interacción con sockets que pide la guía, el prototipo de PE-U1 implementará la mensajería directa (S4) con un socket TCP directo: cada mensaje tendrá un formato fijo (`message_id`, `sequence_no` y contenido) y el servidor responderá con un ACK. La consulta de seguidores (S3) se implementará con gRPC. Así se podrán comparar en la práctica la latencia

y el tamaño de los mensajes de las dos formas. En producción, S4 seguirá usando WSS, porque el navegador no permite abrir sockets TCP directos y WSS viaja cifrado por el mismo puerto que HTTPS.

3.5. Mensajería directa, presencia y moderación

Las cifras de esta sección (500 ms, 1 min, 99,5 %) son *objetivos de diseño*; se comprobarán con pruebas en PE-U1.

Recorrido de un mensaje directo. Al abrir la aplicación, el cliente (CLI) establece un canal WebSocket seguro (WSS sobre TCP) con el balanceador, que lo asigna a una de las instancias de MSG (S4). El canal queda abierto en ambos sentidos. Un mensaje pasa por tres tramos con sincronía distinta:

1. **Envío y ACK de guardado (síncrono para el remitente).** El cliente genera un `message_id` único y envía el mensaje; MSG lo guarda en PostgreSQL (S6) y solo entonces responde al remitente con el **ACK de guardado**. El remitente espera ese ACK y, si no llega, reenvía con el mismo `message_id`.

2. **Entrega al destinatario (asíncrona).** Después de guardar, MSG empuja el mensaje por el canal WSS del destinatario si está conectado. El ACK al remitente no espera a esta entrega.

3. **ACK de entrega (asíncrono).** El cliente del destinatario confirma la recepción con el `sequence_no` recibido; MSG registra ese valor como el último confirmado para ese destinatario en esa conversación.

Por tanto, el **ACK al remitente significa "guardado en PostgreSQL", no "entregado"**. La entrega se confirma aparte con el ACK de entrega del destinatario.

Presencia en línea. Mientras el canal está abierto, el cliente envía periódicamente un *heartbeat* por el mismo canal WSS (S5); no es una petición que bloquee al cliente, por lo que se trata como asíncrono y periódico. Con cada uno, MSG renueva en Redis la clave de presencia del usuario, que tiene un tiempo de expiración (TTL) (S7). Si los *heartbeats* dejan de llegar, la clave expira y el usuario se muestra como desconectado, sin necesidad de un aviso explícito.

Orden: asignación de `sequence_no` entre varias instancias de MSG. Como hay varias instancias, el número de secuencia no puede asignarse en memoria. Se asigna en PostgreSQL, que es el punto común: cada conversación tiene un contador y, dentro de una misma transacción, MSG incrementa ese contador y guarda el mensaje con el valor obtenido. El bloqueo de la fila del contador hace que dos instancias que reciben mensajes de la misma conversación al mismo tiempo se ordenen una detrás de otra y no puedan obtener el mismo número. La tabla de mensajes impone que el par (`conversation_id`, `sequence_no`) y el `message_id` sean únicos.

Deduplicación y cursor de recuperación. Antes de asignar un número, MSG busca el `message_id`: si ya existe (por ejemplo, un reintento tras perder el ACK), devuelve el `sequence_no` ya asignado y no crea otro mensaje ni avanza el contador. Para recuperarse, el cliente conserva por conversación el último `sequence_no` confirmado y, al reconectarse (F1 o F3), lo envía como cursor; MSG responde con los mensajes de esa conversación cuyo `sequence_no` es mayor, en orden. Si el cliente detecta un salto en la numeración, pide el rango que falta.

Denuncia y ocultamiento. Cuando un usuario denuncia un aviso, el cliente envía a MOD el `id_aviso`, el `id_usuario` y el motivo (S8), y MOD confirma la recepción. MOD cuenta únicamente las denuncias válidas de usuarios distintos. Al llegar a tres, solicita a PUB ocultar el aviso (S9); PUB cambia su estado a oculto, que es la fuente de verdad, y publica en Kafka el evento `PublicaciónOcultada` (A4), que MUR consume para retirar o invalidar el aviso en los muros.

Mecanismos y justificación.

- **WSS sobre TCP, bidireccional y persistente (S4, S5).** Un canal abierto permite empujar el mensaje al destinatario apenas se guarda, sin abrir una conexión por cada envío, y sirve también para el *heartbeat*. TCP garantiza entrega ordenada y confiable; WSS cifra el tráfico.
- **PostgreSQL síncrono antes del ACK (S6).** Un mensaje no se confirma al remitente hasta estar guardado, para no confirmar algo que luego pueda perderse.
- **Redis con TTL (S7).** La presencia es un dato temporal que cambia constantemente y no necesita sobrevivir a un reinicio, por lo que se guarda en memoria con expiración.
- **REST y gRPC en la moderación (S8, S9) y Kafka en la invalidación (A4).** La denuncia necesita confirmación inmediata; la orden de ocultar es una llamada directa entre servicios internos; y el retiro del aviso en los muros puede completarse de forma asíncrona.

R2: objetivo de 500 ms y orden. El canal WSS ya está abierto, por lo que la entrega a un destinatario conectado no depende de establecer conexiones nuevas y la tarea más costosa de la ruta es una escritura en PostgreSQL. El `sequence_no` asignado en PostgreSQL mantiene el orden por conversación aunque haya varias instancias, y el `message_id` único evita duplicados. Se plantea como objetivo y se medirá en PE-U1.

R5: objetivo de ocultar en menos de un minuto. La solicitud a PUB (S9) es síncrona y cambia el estado de inmediato. El evento A4 es asíncrono, así que por sí solo no garantiza el límite: si el evento aún no se consumió, MUR podría seguir mostrando su copia del aviso. Para cerrar ese hueco, MUR no se fía solo del evento: al armar el muro para un usuario, consulta a PUB en lote el estado de los avisos que va a mostrar, con una caché de pocos segundos, y descarta los ocultos. Así, tras el cambio de estado en PUB, el aviso deja de verse en cuanto expira esa caché, y el evento A4 se encarga después de limpiar los muros ya armados. Se plantea como objetivo y se medirá en PE-U1.

R6: objetivo de disponibilidad de 99,5 % en la mensajería. Se aspira a esa meta con varias instancias de MSG detrás del balanceador, *health checks* que retiran las instancias que no responden y persistencia de los mensajes en PostgreSQL. El muro está desacoplado mediante Kafka, de modo que su degradación no bloquea la mensajería.

F1: el destinatario está desconectado. El mensaje ya se guardó en PostgreSQL antes del ACK de guardado, por lo que queda pendiente aunque el destinatario no esté en línea. Al reconectarse, el cliente envía su cursor (último `sequence_no` confirmado por conversación) y MSG le entrega los mensajes posteriores.

F3: MSG se reinicia con conversaciones abiertas. Los clientes se reconectan al balanceador, que los asigna a una instancia disponible, y recuperan desde su cursor, sin depender de memoria volátil. Los mensajes que el remitente no vio confirmados se reenvían con el mismo `message_id`, y la deduplicación evita que se guarden dos veces.

4. Protocolos y middleware

En Vecindario se utilizan diferentes protocolos y herramientas para que los servicios puedan intercambiar información. La elección depende del tipo de operación y de la rapidez con la que se necesita procesar cada solicitud.

4.1. Protocolos seleccionados

- **HTTPS:** se utiliza para las solicitudes realizadas desde la aplicación móvil o web hacia los servicios de la plataforma. Permite intercambiar información de manera segura.
- **TCP:** proporciona una conexión confiable para el intercambio de datos. Se utiliza como base de transporte para la comunicación de mensajería.
- **WSS:** permite mantener conexiones bidireccionales y persistentes entre los clientes y el servicio de mensajería, facilitando la entrega de mensajes en tiempo real.
- **HTTP/2:** es utilizado por gRPC para transportar las solicitudes y respuestas entre los servicios de muro y seguidores.

4.2. Middleware y herramientas seleccionadas

- **gRPC:** facilita la comunicación mediante llamadas a procedimientos remotos entre los servicios. Se utiliza principalmente para consultar la información de seguidores desde el servicio de muro.
- **Apache Kafka:** funciona como intermediario de mensajes y permite distribuir eventos entre los servicios de publicaciones, muro y notificaciones sin que el servicio emisor tenga que esperar a todos los consumidores.
- **PostgreSQL:** se propone como sistema de almacenamiento persistente para conservar los mensajes y sus números de secuencia. Además, su mecanismo Write-Ahead Logging (WAL) permite recuperar cambios registrados ante determinados fallos del sistema [4].

- **Redis:** se utiliza para mantener información temporal sobre la presencia de los usuarios, utilizando tiempos de expiración (TTL)[5].
- **Firebase Cloud Messaging (FCM):** se propone para enviar notificaciones push a los dispositivos de los usuarios cuando se generan nuevos avisos o alertas [6].

La combinación de estas tecnologías permite que cada componente utilice el mecanismo más adecuado para su función. De esta manera, las operaciones que necesitan respuestas inmediatas se mantienen separadas de aquellas que pueden procesarse en segundo plano.

5. Tabla de interacciones

La siguiente tabla resume las comunicaciones representadas en el diagrama de Vecindario. Se identifican los servicios que participan, la información intercambiada, el mecanismo utilizado y la razón por la que se seleccionó cada forma de comunicación.

ID	Emisor	Receptor	Mensaje u operación	Tipo	Sincronía	Protocolo / middleware	Justificación
S1	CLI	PUB	Crear aviso con texto, categoría, ubicación y tipo.	Petición-respuesta	Síncrona	REST API / HTTPS sobre TCP	El cliente necesita recibir el identificador del aviso y la confirmación de su creación.
S2	CLI	Almacenamiento externo	Cargar archivo de imagen y recibir su URL o identificador.	Petición-respuesta	Síncrona	API REST / HTTPS sobre TCP	Se necesita obtener la referencia de la imagen antes de vincularla con la publicación.
S2b	CLI	PUB	Registrar el aviso utilizando la URL o identificador de la imagen.	Petición-respuesta	Síncrona	REST API / HTTPS sobre TCP	Permite asociar la imagen previamente almacenada con el aviso correspondiente.
S3	MUR	SEG	Consultar los seguidores del autor mediante páginas o lotes.	RPC	Síncrona	gRPC / HTTP/2	MUR necesita obtener los identificadores de los seguidores para distribuir las publicaciones.
S4	CLI	MSG, mediante el balanceador	Establecer la conexión de mensajería y enviar o recibir mensajes.	Sockets	Síncrona	WebSocket seguro (WSS) / TCP	Permite establecer una conexión persistente y bidireccional para intercambiar mensajes entre el cliente y el servicio de mensajería.
S5	CLI	MSG, mediante el balanceador	Enviar heartbeat y actualizar el estado de conexión.	Sockets	Asíncrona	WSS / TCP	Permite mantener actualizada la presencia de los usuarios mediante mensajes periódicos sin bloquear el intercambio de mensajes.

ID	Emisor	Receptor	Mensaje u operación	Tipo	Sincronía	Protocolo / middleware	Justificación
S6	MSG	PostgreSQL	Guardar mensaje, conversación, identificador y número de secuencia.	Acceso a datos	Síncrona	Protocolo PostgreSQL / TCP	El mensaje debe quedar almacenado antes de confirmar su recepción para evitar pérdidas.
S7	MSG	Redis	Actualizar o consultar la presencia de los usuarios mediante TTL.	Acceso a datos	Síncrona	RESP / TCP	Permite consultar rápidamente el estado temporal de conexión y detectar desconexiones mediante expiración.
S8	CLI	MOD	Enviar denuncia con identificador del aviso, usuario y motivo.	Petición-respuesta	Síncrona	REST API / HTTPS sobre TCP	MOD necesita recibir la denuncia y confirmar su recepción para iniciar la revisión.
S9	MOD	PUB	Solicitar el ocultamiento de una publicación al alcanzar tres denuncias válidas.	RPC	Síncrona	gRPC / HTTP/2	PUB debe recibir la solicitud y cambiar el estado del aviso a oculto.
S10	CLI	MUR	Consultar el muro mediante una solicitud de timeline paginado.	Petición-respuesta	Síncrona	REST API / HTTPS sobre TCP	Permite consultar las publicaciones del usuario, mostrando primero las alertas.
A1	PUB	Kafka	Publicar el evento AvisoPublicado con los datos del aviso.	Mensajería pub/sub	Asíncrona	Kafka / TCP	PUB puede continuar su operación sin esperar a que todos los consumidores procesen el evento.
A2	Kafka	MUR	Consumir el evento AvisoPublicado para distribuirlo en los muros.	Mensajería pub/sub	Asíncrona	Kafka / TCP	Permite procesar las publicaciones en segundo plano y distribuirlas por lotes.

ID	Emisor	Receptor	Mensaje u operación	Tipo	Sincronía	Protocolo / middleware	Justificación
A3	Kafka	NOT	Consumir eventos NuevaPublicación o AlertaSeguridad.	Mensajería pub/sub	Asíncrona	Kafka / TCP	NOT recibe los eventos sin bloquear al servicio que los genera y puede gestionar las notificaciones.
A4	PUB	Kafka / MUR	Publicar PublicaciónOcultada y consumir el evento para invalidar el aviso en los muros.	Mensajería pub/sub	Asíncrona	Kafka / TCP	Permite comunicar el ocultamiento sin esperar a que todos los muros actualicen su contenido.
A5	NOT	CLI	Enviar notificación push de nuevo aviso o alerta de seguridad.	Mensajería push	Asíncrona	Firebase Cloud Messaging (FCM) / HTTPS sobre TCP [6]	Permite avisar al usuario sin que tenga que consultar constantemente la plataforma.

6. Requisitos y situaciones de fallo

El diseño de Vecindario debe responder a las condiciones establecidas para la plataforma, especialmente a la cantidad de usuarios conectados y a la rapidez con la que deben procesarse las publicaciones y los mensajes. También se consideran posibles fallos para evitar que la información se pierda cuando alguno de los servicios deja de funcionar.

6.1. Cumplimiento de los requisitos

- **R1. Cantidad de usuarios:** la plataforma debe soportar 200.000 usuarios registrados y hasta 40.000 conectados simultáneamente. Para atender esta demanda se propone utilizar un balanceador de conexiones y varias instancias de los servicios que necesiten distribuir su carga.
- **R2. Mensajería directa:** los mensajes deben llegar en menos de 500 milisegundos cuando el destinatario está conectado y conservar el orden en que fueron enviados. Para ello, MSG utilizará conexiones persistentes mediante WSS, números de secuencia por conversación y almacenamiento de los mensajes antes de confirmar su recepción.
- **R3. Tiempo de distribución:** las publicaciones normales pueden tardar hasta cinco segundos en aparecer en el muro, mientras que las alertas de seguridad deben aparecer en menos de un segundo. Para conseguirlo se propone utilizar Kafka y dar prioridad al procesamiento de las alertas.
- **R4. Negocios con muchos seguidores:** cuando un negocio tiene 30.000 seguidores, su publicación se distribuirá por lotes para evitar que todas las solicitudes se procesen al mismo tiempo. De esta manera se busca reducir la carga sobre el servicio de muro.
- **R5. Moderación de publicaciones:** cuando una publicación recibe tres denuncias, MOD debe gestionar su ocultamiento mediante PUB. El cambio también debe comunicarse al servicio de muro para que deje de mostrar el contenido, procurando cumplir el límite de un minuto.
- **R6. Disponibilidad:** el servicio de mensajería debe alcanzar una disponibilidad del 99,5 %. Para ello se propone utilizar varias instancias de MSG, supervisar su funcionamiento y conservar los mensajes en PostgreSQL. El servicio de muro podrá presentar una degradación sin que esto interrumpa necesariamente la mensajería.

6.2. Respuesta ante situaciones de fallo

- **F1. Destinatario desconectado:** si el destinatario no está conectado, el mensaje se conserva en PostgreSQL y queda pendiente de entrega. Cuando el usuario vuelve a conectarse, MSG recupera los mensajes que todavía no han sido confirmados y los envía.
- **F2. Caída del servicio de muro:** si MUR deja de funcionar mientras se distribuye una publicación de un negocio con 30.000 seguidores, Kafka conserva los eventos pendientes. Cuando el servicio se recupera, puede continuar con su procesamiento.

Además, se deben utilizar mecanismos de idempotencia para evitar que una misma publicación se procese varias veces.

- **F3. Reinicio del servidor de mensajería:** si MSG se reinicia mientras existen conversaciones abiertas, los clientes pueden volver a conectarse. El servicio recupera los mensajes pendientes a partir del último número de secuencia confirmado. Los identificadores únicos permiten reconocer los mensajes que ya fueron procesados y reducir las duplicaciones.

Estas medidas buscan que la plataforma pueda continuar funcionando ante interrupciones y que los usuarios no pierdan sus mensajes ni la información de sus publicaciones.

7. Preguntas críticas recibidas y respuestas

A la fecha de elaboración de este documento, el equipo todavía no ha recibido preguntas críticas durante la revisión en clase. Este apartado se completará cuando se realicen dichas intervenciones.

8. Preguntas críticas formuladas a otros equipos

A la fecha de elaboración de este documento, el equipo todavía no ha formulado preguntas a otros grupos. Este apartado se completará cuando se realice la revisión de sus trabajos.

9. Retroalimentación y ajustes

Esta sección se completará cuando el docente proporcione sus observaciones sobre el diseño de Vecindario. Se registrarán las recomendaciones recibidas y los cambios realizados por el equipo para mejorar la propuesta.

10. Declaración de uso de inteligencia artificial

Durante la elaboración de este trabajo se utilizó ChatGPT como herramienta de apoyo para organizar contenidos, mejorar la redacción, revisar explicaciones técnicas y estructurar parte de la documentación.

La herramienta no sustituyó el trabajo del equipo en la elaboración del diagrama ni en la toma de decisiones sobre la arquitectura. Los integrantes revisaron el contenido y son responsables de la información incluida en el documento.

Referencias

[1] Fette, I., y Melnikov, A. (2011). *The WebSocket Protocol (RFC 6455)*. Internet Engineering Task Force. `https://www.rfc-editor.org/rfc/rfc6455`

[2] gRPC Authors. (s. f.). *Introduction to gRPC*. `https://grpc.io/docs/what-is-grpc/introduction/`

[3] Apache Software Foundation. (s. f.). *Apache Kafka Documentation*. https://kafka.apache.org/documentation/

[4] PostgreSQL Global Development Group. (s. f.). *Write-Ahead Logging (WAL)*. https://www.postgresql.org/docs/current/wal-intro.html

[5] Redis Ltd. (s. f.). *EXPIRE Command*. https://redis.io/docs/latest/commands/expire/

[6] Google. (s. f.). *Send a message using FCM HTTP v1 API*. https://firebase.google.com/docs/cloud-messaging/send/v1-api